

LABBGEN

LLM-based Automated Black-Box
Test Generator

Tommaso Fulcini



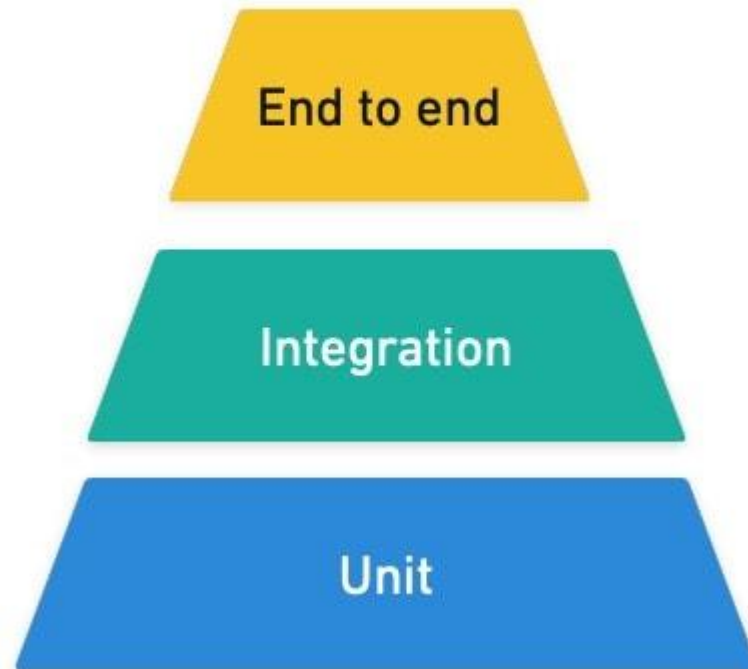
SWICh projects 2024



Operational software failure can cost up to **10 times more** than defects identified before the release

Creating a **robust test suite** is crucial for industrial software

The Test Pyramid



Guidelines suggest that Software should be **tested at different levels**

The lower the level, the more tests should be written

End-to-end automated tests tend to be overlooked



Some companies completely **outsource** end-to-end testing

For external companies, building robust and correct test suites is a matter of communication and documentation



In some cases, the documentation or requirements may be of **poor quality** or, even more often, **out of date**

Dialoging with the software owner sometimes can be exhausting and redundant



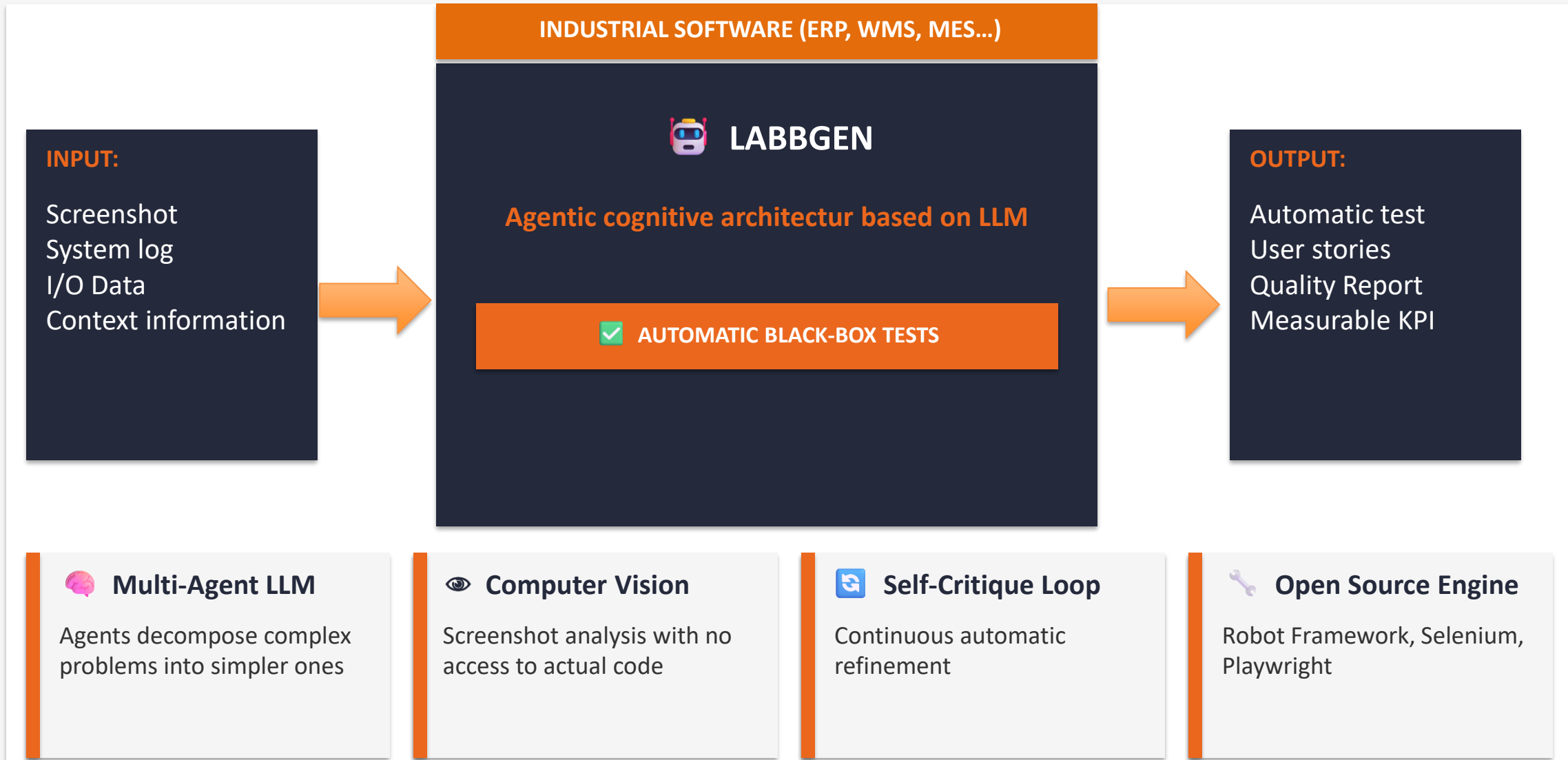
Agentic AI is an artificial intelligence system that can **accomplish** a specific **goal** with **limited supervision**, according to IBM

Agents have access to a set of tools that can be invoked thanks to the reasoning of LLMs

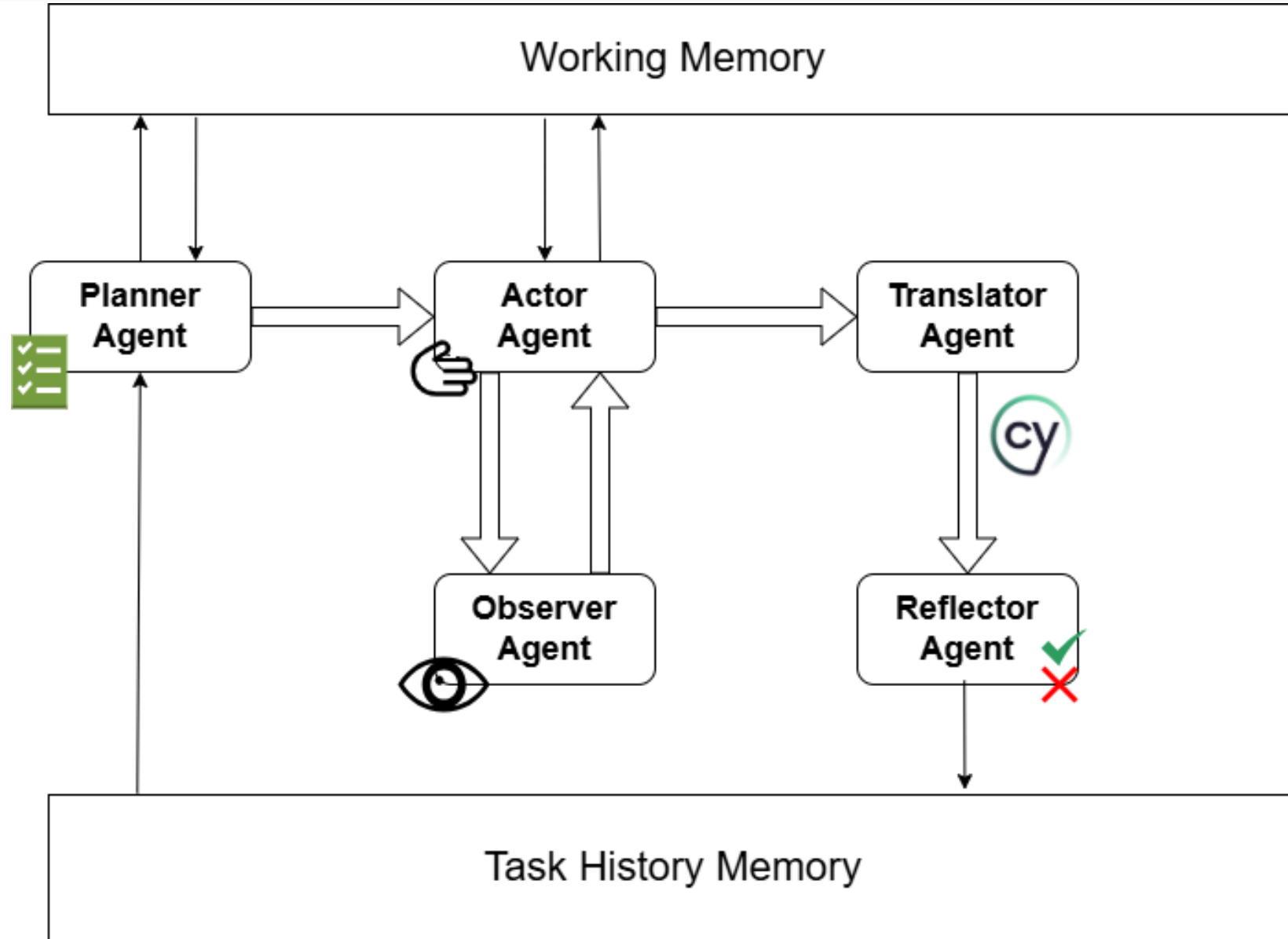
[3] IBM: <https://www.ibm.com/think/topics/agentic-ai>

[4] Amalfitano et al. *A Research Roadmap for Augmenting Software Engineering Processes and Software Products with Generative AI*, TOSEM (2026)

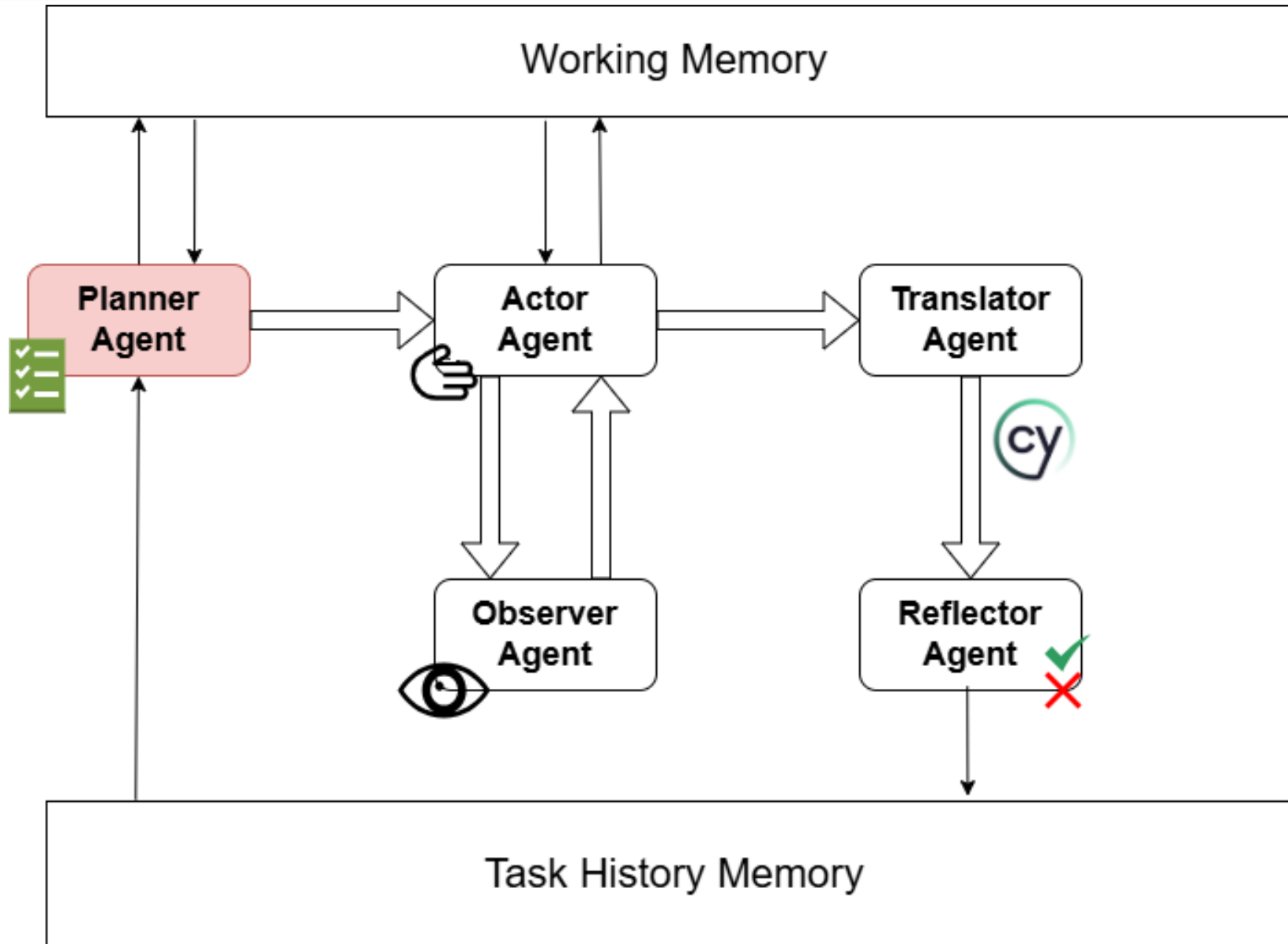
LABBGEN : The proposed solution



Our agentic architecture proposal



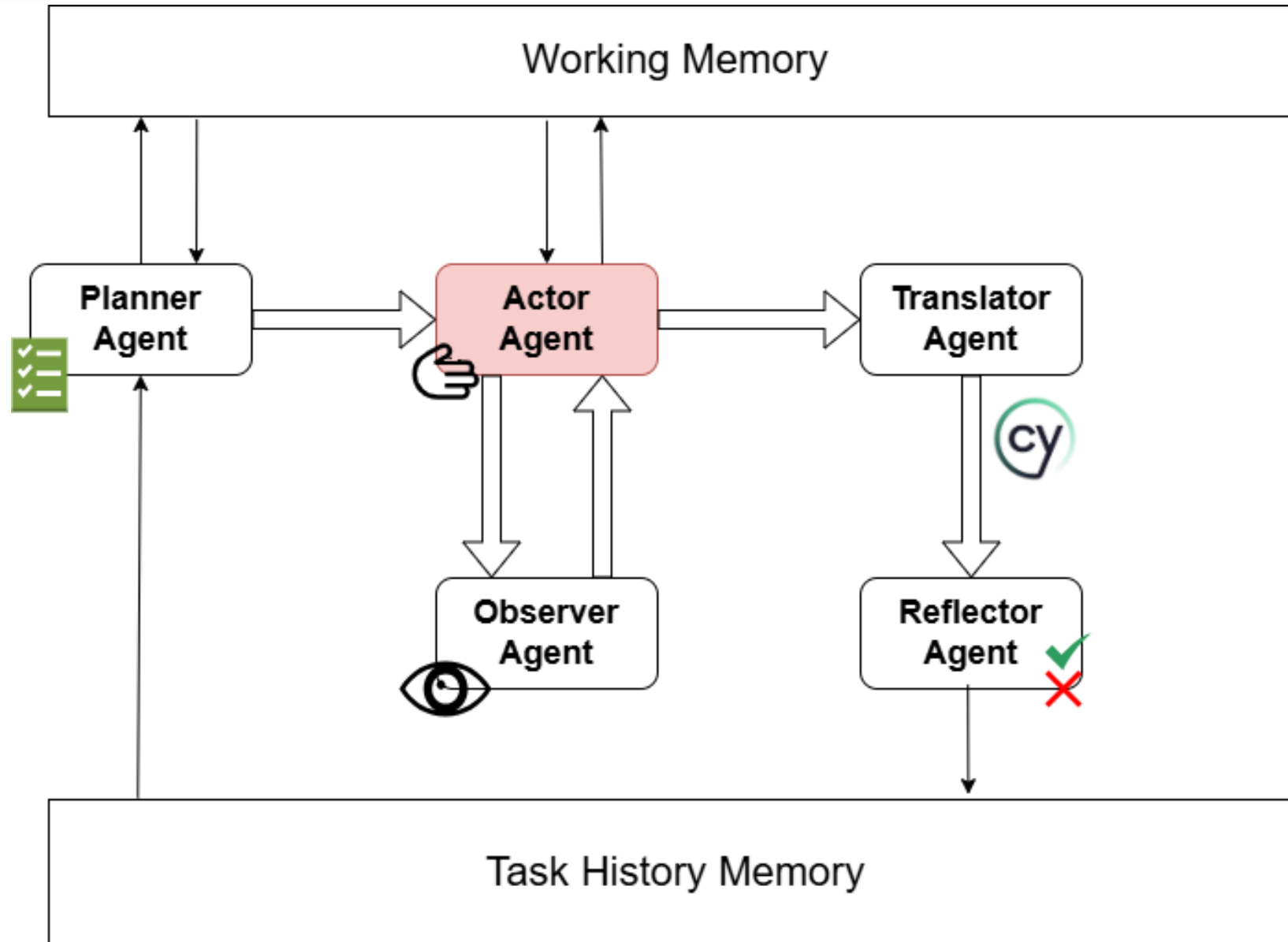
Our agentic architecture proposal



Planner:

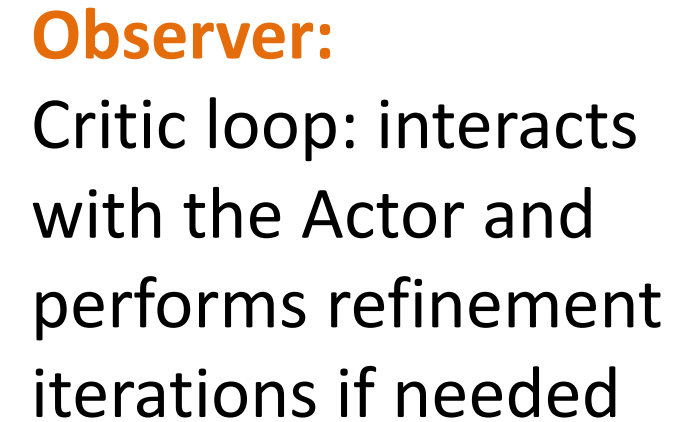
Plan high-level goals.
Interact with
memories to focus on
tasks

Our agentic architecture proposal

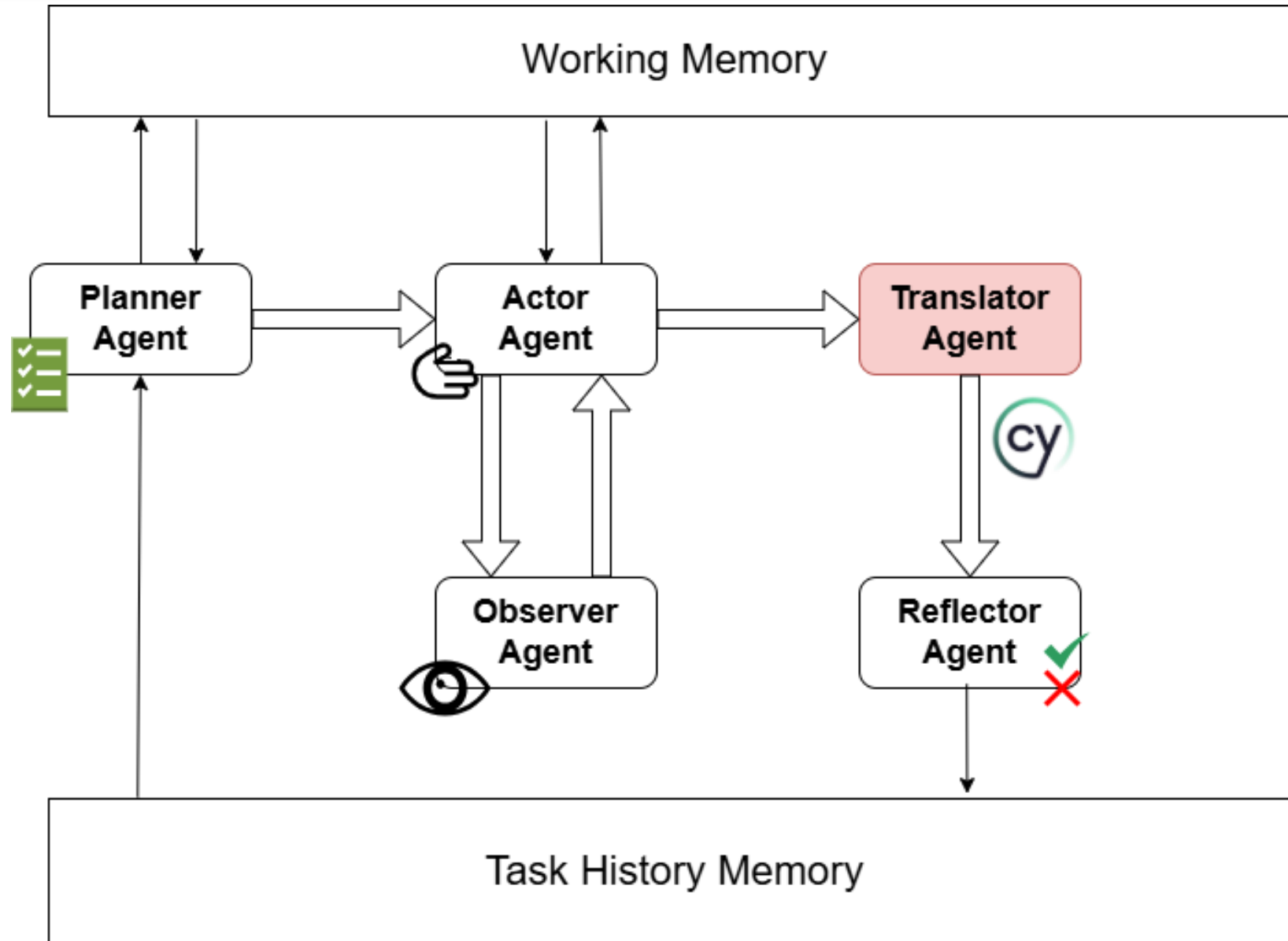


Actor:

Generate detailed user stories based on the documentation and screenshots that have been analyzed



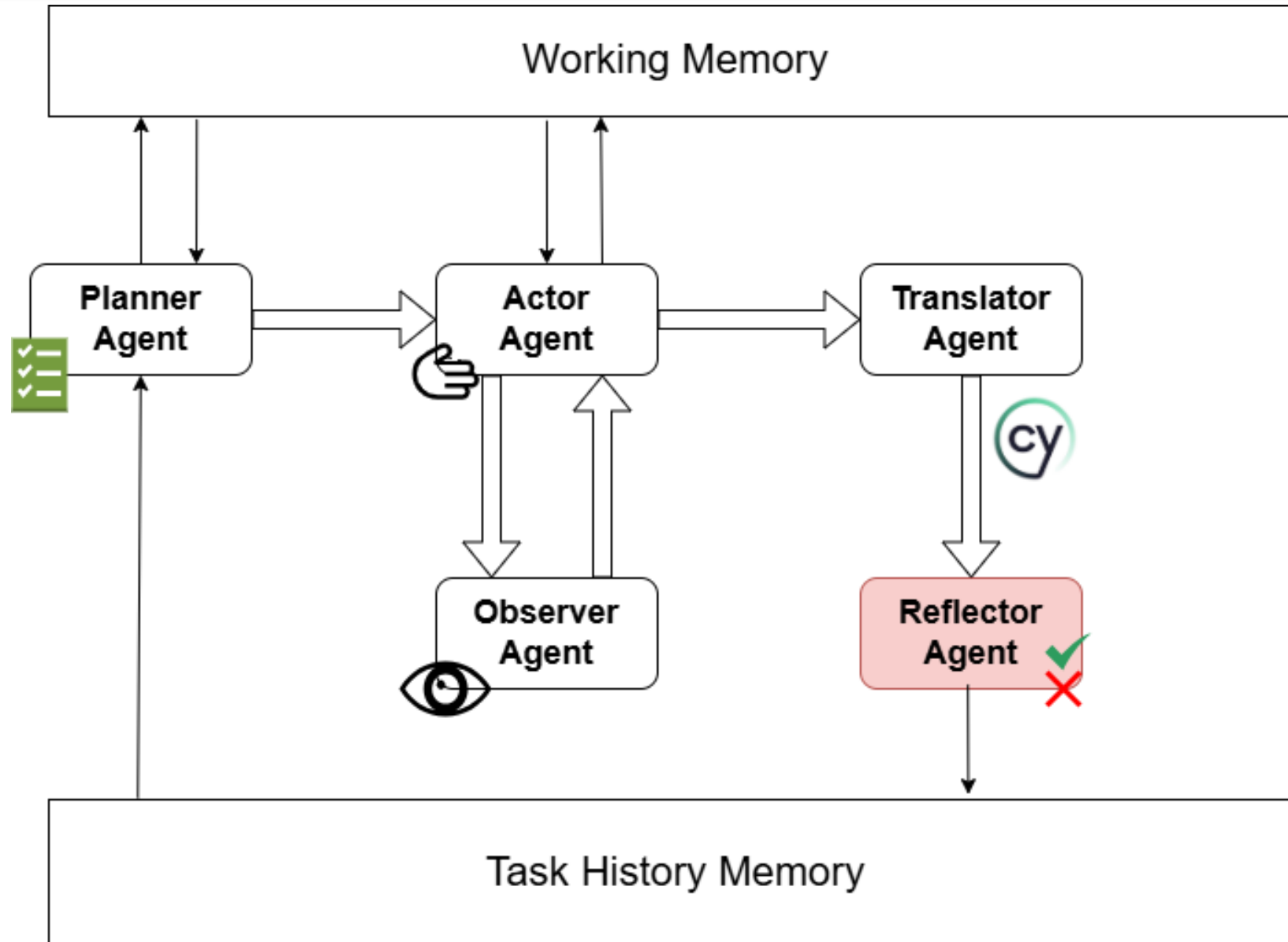
Our agentic architecture proposal



Translator:

Translates the received user story into Cypress code for execution

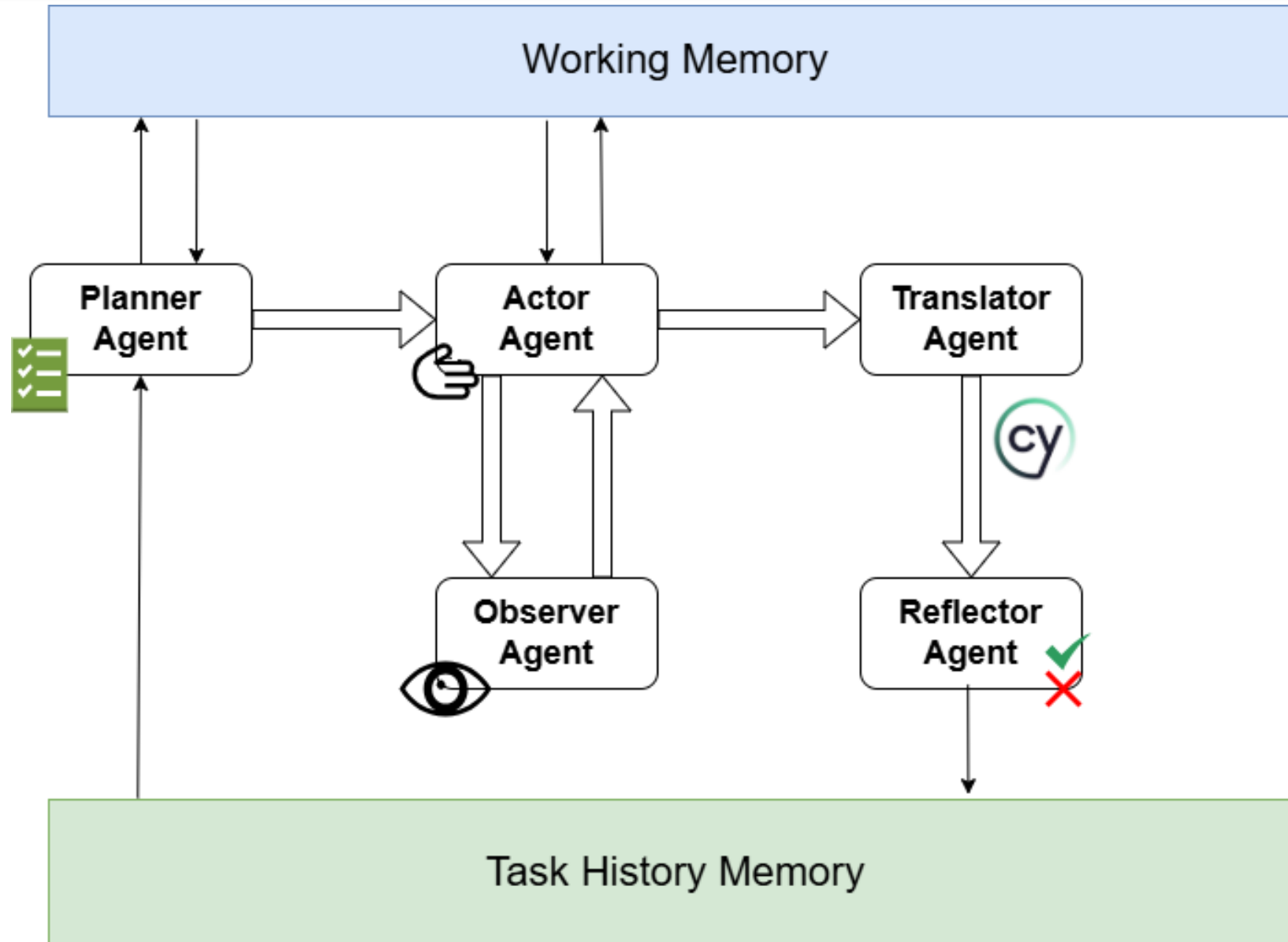
Our agentic architecture proposal



Reflector:

It runs the Cypress test, analyzes the result and saves it to memory

Our agentic architecture proposal



Working Memory:

Current task status, feedback, and application input and output

Task History Memory:

History of tasks, results, and patterns for future reuse

1

A technology-agnostic approach

Generate high-level pseudocode tests before translating them into the specific framework (Selenium, Playwright...). It works on desktop, web, and mobile without any architectural changes.

2

Automatic self-criticism loop

The Observer analyzes each output and drives improvement cycles until the result meets the defined formal and logical criteria. It reduces hallucinations in LLM models.

3

Synchronization with documentation

The metadata used can enrich existing documentation and generate new documentation where none exists.

4

Selective Human-in-the-loop

Users can choose between step-by-step supervision and on-demand intervention. Domain experts guide the system without having to write test code.

What are the current problems?



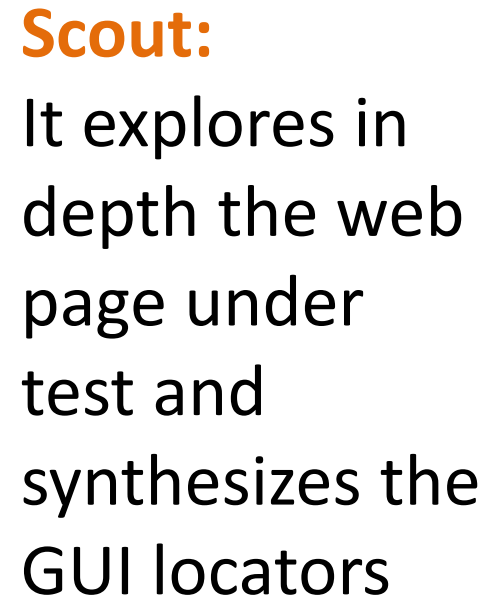
**Behaviour
Misalignment**



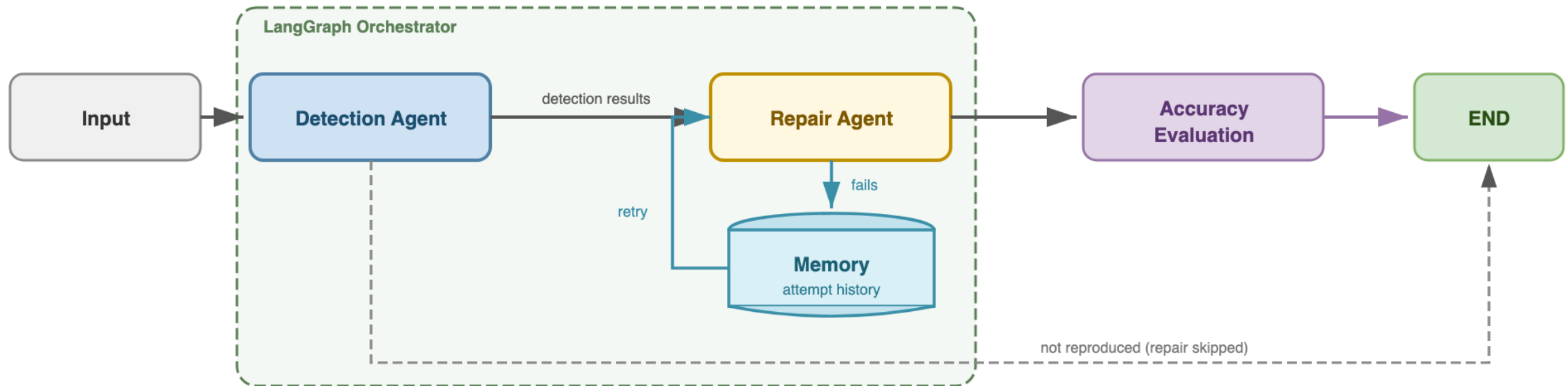
Getting stuck



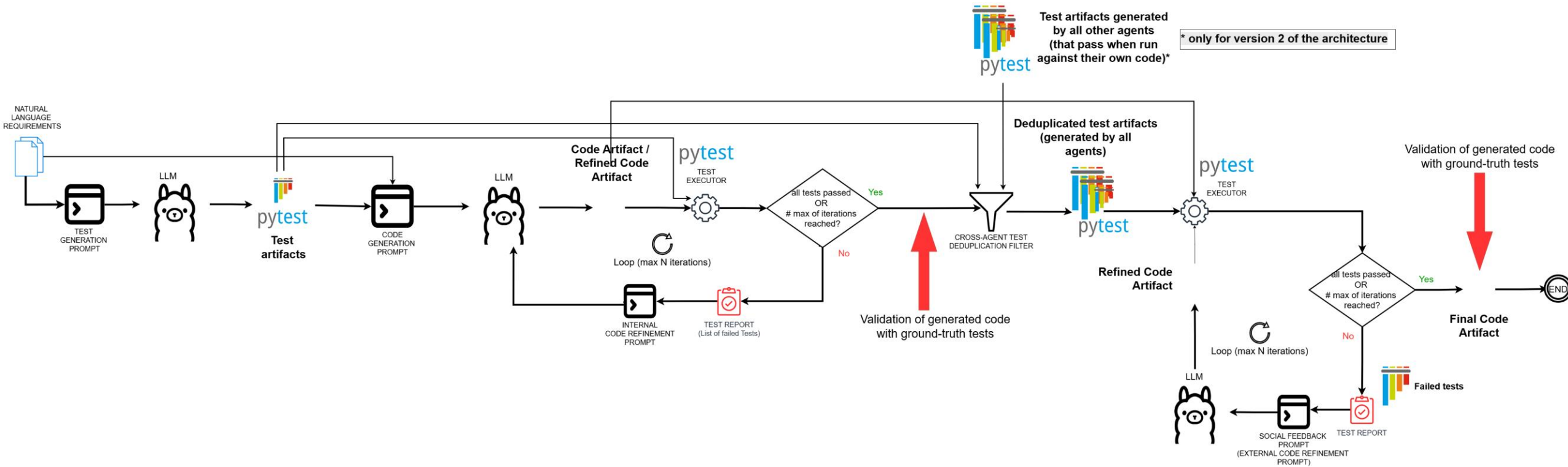
Confirmation Bias



HA-FlakyRepair — Architecture Overview



Other Derivations: Diversity-based unit test generation





**Continue to Keep
It Simple, Stupid**



**Make Human
great again**



Context Matters!

Thank you!



Stay in touch with me :)

Ask



**Politecnico
di Torino**